FlightFinder - Flight Booking Application

Project Documentation

Table of Contents

1. Project Overview
2. Features
3. Technology Stack
4. System Architecture
5. Installation Guide
6. User Roles
7. API Endpoints
8. Database Schema
9. Key Features Implementation
10. Security Features

Project Overview

FlightFinder is a comprehensive flight booking web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js). The application provides a seamless experience for customers to search and book flights, flight operators to manage their flights, and administrators to oversee the entire system.

Key Objectives

- Provide an intuitive interface for flight booking
- Enable flight operators to manage their flight schedules
- Allow administrators to control and monitor the platform
- Ensure secure authentication and authorization
- Prevent booking of past dates

Features

Customer Features

- **Flight Search:** Search flights by departure city, destination, and travel dates
- **Date Validation:** Past dates are automatically disabled
- **Seat Class Selection:** Choose from Economy, Premium Economy, Business, or First Class
- **Multiple Passengers:** Book tickets for multiple passengers in one transaction
- **Dynamic Pricing:** Real-time price calculation based on seat class and number of passengers
- **Booking Management:** View all bookings and cancel tickets
- **Secure Authentication:** Login and registration with encrypted passwords

Flight Operator Features

- **Registration with Approval:** Register and wait for admin approval
- **Flight Management:** Add new flights with complete details
- **Edit Flights:** Update flight information (departure time, price, seats, etc.)
- **View Bookings:** See all bookings for their flights
- **Dashboard:** Overview of all flights and booking statistics

Admin Features

- **User Management:** View all users and their details
- **Operator Approval:** Approve or reject flight operator registrations
- **Flight Oversight:** View and manage all flights across all operators
- **Booking Monitoring:** Track all bookings in real-time
- **Complete Control:** Full administrative access to the platform

Technology Stack

Frontend

- **React.js** - UI library for building interactive interfaces
- **React Router** - Client-side routing
- **Axios** - HTTP client for API requests
- **Bootstrap 5** - CSS framework for responsive design
- **Context API** - State management

Backend

- **Node.js** - JavaScript runtime environment
- **Express.js** - Web application framework
- **MongoDB** - NoSQL database
- **Mongoose** - MongoDB object modeling
- **bcrypt** - Password hashing
- **CORS** - Cross-origin resource sharing

System Architecture

```
┌─────────────────┐
│   React Client  │
│   (Port 3000)   │
└─────────────────┘
         │
         │
         │ HTTP Requests
         │
┌────────▼────────┐
│  Express Server │
│   (Port 6001)   │
└─────────────────┘
         │
         │
         │ Mongoose
         │
┌────────▼────────┐
│    MongoDB      │
│   (Port 27017)  │
└─────────────────┘
```

Installation Guide

Prerequisites

- Node.js (v14 or higher)
- MongoDB (v4.4 or higher)
- npm or yarn package manager

Installation Steps

1. **Clone the Repository**

```
git clone <repository-url>
cd "FlightFinder Navigating Your Air Travel Options"
```

2. **Install Server Dependencies**

```
cd server
npm install
```

3. **Install Client Dependencies**

```
cd ../client
npm install
```

4. **Start MongoDB**

```
net start MongoDB
```

5. **Start the Server**

```
cd server
npm start
# Server runs on http://localhost:6001
```

6. **Start the Client**

```
cd client
npm start
# Client runs on http://localhost:3000
```

User Roles

1. Customer

- **Registration:** Instant approval
- **Capabilities:** Search flights, book tickets, manage bookings
- **Access Level:** Limited to own bookings

2. Flight Operator

- **Registration:** Requires admin approval
- **Capabilities:** Add/edit flights, view flight bookings
- **Access Level:** Access to own flights only

3. Administrator

- **Registration:** Manual registration as admin
- **Capabilities:** Full system control
- **Access Level:** Complete access to all data

API Endpoints

Authentication

Method	Endpoint	Description
POST	/register	Register new user
POST	/login	User login

User Management

Method	Endpoint	Description
GET	/fetch-user/:id	Get user by ID
GET	/fetch-users	Get all users
POST	/approve-operator	Approve flight operator
POST	/reject-operator	Reject flight operator

Flight Management

Method	Endpoint	Description
POST	/add-Flight	Add new flight
PUT	/update-flight	Update flight details
GET	/fetch-flights	Get all flights
GET	/fetch-flight/:id	Get flight by ID

Booking Management

Method	Endpoint	Description
POST	/book-ticket	Book a flight ticket
GET	/fetch-bookings	Get all bookings
PUT	/cancel-ticket/:id	Cancel booking

Database Schema

User Schema

```
{
  username: String (required),
  email: String (required, unique),
  usertype: String (required),
  password: String (required, hashed),
  approval: String (default: 'approved')
}
```

Flight Schema

```
{
  flightName: String (required),
  flightId: String (required),
  origin: String (required),
  destination: String (required),
  departureTime: String (required),
  arrivalTime: String (required),
  basePrice: Number (required),
  totalSeats: Number (required)
}
```

Booking Schema

```
{
  user: ObjectId (ref: User),
  flight: ObjectId (ref: Flight),
  flightName: String,
  flightId: String,
  departure: String,
  destination: String,
  email: String,
  mobile: String,
  seats: String,
  passengers: [{
    name: String,
    age: Number
  }],
  totalPrice: Number,
  bookingDate: Date (default: now),
  journeyDate: Date,
  journeyTime: String,
  seatClass: String,
  bookingStatus: String (default: 'confirmed')
}
```

Key Features Implementation

1. Date Validation

Past dates are disabled using HTML5 date input min attribute:

```
min={new Date().toISOString().split('T')[0]}
```

2. Dynamic Pricing

Price calculation based on seat class multiplier:

```
const price = {
  'economy': 1,
  'premium-economy': 2,
  'business': 3,
  'first-class': 4
}
totalPrice = price[coachType] * basePrice * numberOfPassengers
```

3. Seat Allocation

Automatic seat assignment with class-based codes:

```
const seatCode = {
  'economy': 'E',
  'premium-economy': 'P',
  'business': 'B',
  'first-class': 'A'
}
```

4. Password Security

Passwords are hashed using bcrypt with salt rounds:

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Security Features

1. **Password Encryption:** bcrypt hashing with salt
2. **Protected Routes:** Authentication required for sensitive pages
3. **CORS Configuration:** Controlled cross-origin requests
4. **Input Validation:** Client and server-side validation
5. **Role-Based Access:** Different permissions for each user type

Future Enhancements

1. **Payment Integration:** Add payment gateway (Stripe/PayPal)
2. **Email Notifications:** Booking confirmations via email
3. **Flight Status:** Real-time flight status updates
4. **Reviews & Ratings:** Customer feedback system
5. **Mobile App:** React Native mobile application
6. **Advanced Search:** Filters for price range, duration, stops
7. **Loyalty Program:** Reward points for frequent flyers
8. **Multi-language Support:** Internationalization

Troubleshooting

Common Issues

1. Registration Failed

- Ensure MongoDB is running
- Check server is running on port 6001

- Verify all fields are filled

2. Server Connection Error

- Start MongoDB service
- Check port 6001 is not in use
- Verify MongoDB connection string

3. Past Dates Selectable

- Clear browser cache
- Ensure latest code is deployed
- Check date input min attribute

© 2026 SB FlightConnect - All rights reserved

View Demo Video